

Complexity analysis of the SAT engine: DNA algorithms as probabilistic algorithms

Masami Hagiya^{a,*}, John A. Rose^a, Ken Komiya^b, Kensaku Sakamoto^b

^a*Department of Computer Science, Graduate School of Information Science and Technology,
The University of Tokyo, Tokyo, Japan*

^b*Department of Biophysics and Biochemistry, Graduate School of Science, The University of Tokyo,
Tokyo, Japan*

Abstract

Taking advantage of the power of DNA molecules to spontaneously form hairpin structures, Sakamoto et al. designed a molecular algorithm to solve instances of the satisfiability problem on Boolean expressions in clausal form (the SAT problem), and by developing new experimental techniques for molecular biology, they succeeded in solving a 6-variable, 10-clause instance of the 3-SAT problem (Sakamoto et al., *Science* 288 (2000) 1223). Sakamoto et al. call this computational architecture the *SAT Engine*. In this paper, we analyze the complexity of the SAT Engine as a probabilistic algorithm. We first estimate the time dependence of the probability of hairpin formation using standard chemical kinetics and the Jacobson–Stockmayer expression. We then estimate the number of DNA molecules required to solve the satisfiability problem with a given error probability. By taking the number of DNA molecules into account, we finally estimate the minimum total time and number of strands, respectively, required to achieve combined error rates of $< \varepsilon_1$ (the probability of a false positive) and ε_2 (the probability of a false negative). If the number of clauses is n , then the time required for solving the problem is proportional to $n^{1.5}(\ln(1/\varepsilon_1) + \ln(\ln(1/\varepsilon_2))) + n^{2.5} \ln(3 + \alpha)$, and the number of necessary DNA molecules is proportional to $(3 + \alpha)^n \ln(1/\varepsilon_2)$ with arbitrarily small $\alpha > 0$. © 2002 Elsevier Science B.V. All rights reserved.

1. Introduction

Taking advantage of the power of DNA molecules to autonomously form hairpin structures, Sakamoto et al. designed a molecular algorithm to solve the satisfiability problem on Boolean expressions in clausal form (the k -SAT problem), and by developing new techniques for experimental molecular biology, they succeeded in solving

* Corresponding author.

a 6-variable, 10-clause instance of the 3-SAT problem [19]. Sakamoto et al. call this computational architecture the *SAT Engine*.

In addition to the SAT Engine, Sakamoto et al. have also proposed a computational architecture for the solution of instances of NP-complete problems, called the *Whiplash Engine* (or *Whiplash PCR*) [8,20]. An enhanced-efficiency version of this architecture, PNA-mediated Whiplash PCR [16] has been shown to be theoretically capable of supporting the in vitro implementation of an extraction-based Genetic Algorithm [17]. Because these architectures (referred to collectively as the *Hairpin Engines*) take advantage of the tendency for an appropriately encoded single-stranded DNA molecule (ssDNA) to spontaneously adopt a DNA hairpin structure under appropriate reaction conditions, the study on the Hairpin Engines represents a first step towards establishing architectures that harness secondary structure formation to achieve computation.

The SAT Engine is an algorithm for solving the satisfiability problem on Boolean expressions in clausal form [19]. Since a Boolean expression in clausal form is a conjunction of clauses, an expression is satisfiable if and only if all the conjuncts are simultaneously satisfiable. Since a clause is a disjunction of literals, a clause is satisfiable if and only if at least one of the literals is satisfiable. An expression is therefore satisfiable if and only if it is possible to make an assignment by consistently choosing one literal from each clause. For example, if we choose the literal a from the clause $(a \vee \neg b \vee c)$, we make a true in the corresponding assignment. If we choose the literal $\neg b$, we make b false. If we simultaneously choose a variable (say b) and its negation (say $\neg b$), the literal selection is inconsistent and we cannot make an assignment. Conversely, if we do not choose inconsistent literals, we can make an assignment that satisfies the given Boolean expression.

In the SAT Engine, each variable and its negation are represented by complementary DNA sequences in the sense of Watson and Crick. Prior to the computation, we randomly generate a set of ssDNA molecules, each of which encodes a set of literals selected from the clauses of the Boolean expression of interest. In particular, each DNA molecule is encoded to represent a concatenation of sequences of literals, such that one literal is selected from each clause contained in the given Boolean expression. If the literal selection is inconsistent (i.e., if a variable and its negation are simultaneously chosen), then under appropriate reaction conditions, the corresponding ssDNA molecule will be able to form a stable hairpin structure, as shown in Fig. 1. If the literal selection is consistent, the DNA molecule will not form a hairpin. After the random generation of assignments (i.e., literal selection), we may therefore judge the satisfiability of a Boolean expression by separating the ssDNAs that form a hairpin from those that do not.

Sakamoto et al. have developed a number of experimental techniques to separate molecules according to their ability to form a hairpin under specific reaction conditions [19]. In the SAT Engine, the following two techniques for separating molecules are employed:

- (1) A restriction site is inserted in the sequences representing literals. Those ssDNAs that form a hairpin are then cut using the corresponding restriction enzyme.
- (2) The efficiency of PCR amplification is different for ssDNAs which contain a hairpin (in the region of interest) and those which do not. Sakamoto has developed a

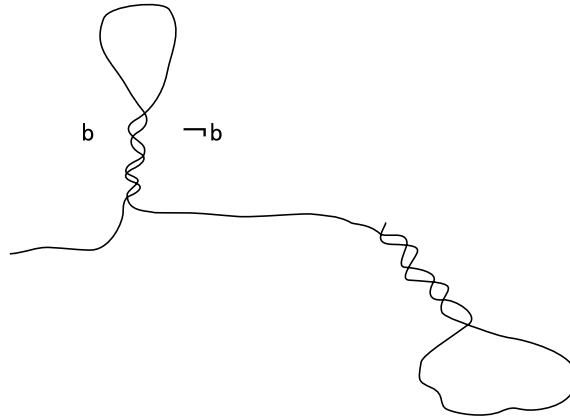


Fig. 1. An inconsistent assignment.

variation of PCR that intensifies this distinction. This variation is called *exclusive PCR* (or *ePCR*).

By application of the above two techniques, Sakamoto et al. succeeded in solving a 6-variable, 10-clause instance of 3-SAT [19]. Although these two techniques require a number of experimental operations, the detection of inconsistent literals within each assignment is implemented by the process of hairpin formation, which is an autonomous, intramolecular reaction. As a consequence, the number of experimental operations which are required to implement computation does not depend directly on either the number of variables or clauses.

As chemical reactions are phenomena that depend on the probabilistic behaviors of molecules, molecular computation is inherently probabilistic. In the SAT Engine, both the reaction in which the set of DNA molecules that encode the set of assignments is randomly generated, and the reaction in which DNA molecules which encode inconsistent assignments form a hairpin proceed probabilistically. Apart from molecular computing, many probabilistic algorithms have been proposed to solve instances of the satisfiability problem on Boolean expressions. For example, Schönning [21] gives a probabilistic algorithm that solves instances of the n -variable k -SAT problem and runs in a time proportional to $(2(1 - 1/k))^n \ln(1/\varepsilon)$ with error probability ε . For $k = 3$, the required time is proportional to $1.334^n \ln(1/\varepsilon)$. It is therefore natural to compare molecular algorithms with traditional probabilistic algorithms. In this work, we analyze the computational complexity of the Sakamoto et al. SAT Engine as a probabilistic algorithm. Briefly, our results are as follows.

The functional form of the equation which predicts the time required for hairpin formation can be used to assess the error probability for any given ssDNA. In particular, in order to ensure that the probability that a ssDNA which represents an inconsistent assignment will fail to form a hairpin is smaller than ε_0 , the SAT Engine requires a time proportional to $n^{1.5} \ln(1/\varepsilon_0)$.

Let the probability that the process of random generation of assignments fails to generate a satisfying assignment, even though such an assignment exists, be denoted

ε_2 . In order to estimate the overall time required to achieve a probability of failure that is less than ε_2 , we must take into account the total number of encoded ssDNAs used in the reaction mixture. The number of molecules required to implement the algorithm is proportional to $(3 + \alpha)^n \ln(1/\varepsilon_2)$, where n is the number of clauses and α is an arbitrarily small positive real number. The time required for the random generation of assignments is proportional to n .

Combining these results yields the expressions

$$t = O\left(n^{1.5} \left(\ln\left(\frac{1}{\varepsilon_1}\right) + \ln\left(\ln\left(\frac{1}{\varepsilon_2}\right)\right)\right) + n^{2.5} \ln(3 + \alpha)\right),$$

and

$$M = O\left((3 + \alpha)^n \ln\left(\frac{1}{\varepsilon_2}\right)\right)$$

for the minimum total time and number of strands, respectively, required to achieve combined error rates of less than ε_1 (the probability of a false positive) and ε_2 (the overall probability of a false negative).

The organization of the paper is as follows. In Section 2, we summarize some of the existing work on the complexity analysis of molecular computation. Section 3 addresses the time for hairpin formation for individual ssDNAs. In Section 4, the scaling behavior of the required number of molecules is addressed, by (1) analyzing the stepwise generation of assignments and (2) discussing the process of one-step generation. In Section 5, the total time and number of molecules required for execution of the SAT Engine with combined error rates of less than ε_1 and ε_2 is estimated.

2. Related work

In this section, we briefly review the existing research into the complexity of molecular computation. After describing some classical results, we discuss approaches which have relied on an analysis of the equilibrium chemistry and/or kinetics of the underlying chemical reactions. Finally, we touch on the possibility of implementing probabilistic algorithms by molecules.

2.1. Time and space in molecular computation

The complexity of any computation should be measured according to the criteria, time and space. Time in molecular computation is usually estimated by the product of the number of experimental operations and the time required for each operation (i.e., the physical time). Unfortunately, as the physical time required to execute an experimental operation is heavily dependent on the actual experimental techniques employed, it is often difficult to make an implementation-independent estimate of the time resource required to execute a given molecular algorithm. The physical time required for each operation is therefore generally left untreated, and the required time resource in molecular computation is simply measured by the total number of required operations.

Nevertheless, in order to investigate the *computational power of a molecular architecture*, it is important to estimate the time required for the set of chemical reactions that constitute an experimental operation. This point will be discussed in detail in the next section.

The space required for molecular computation is generally defined using some measure of the physical amount of necessary molecules (i.e., total mass, volume, etc.). One convenient measure of space is the product of the mass of each molecule and the total number of molecules employed (the total mass). Such a measure is used to estimate either (1) the maximum amount of molecules that are expected to be in solution at any instant during computation, or (2) the total amount of molecules that are consumed in the overall process of computation. Note that the former amount determines the degree of parallelism available in data parallel computation, à la Adleman [1].

Some classical results on the complexity of molecular computation are as follows.

- Reif (1995): Computation in $2^{O(s)}$ steps by a Turing machine whose input size is s and whose space is also s can be simulated by $O(s)$ PM-Match steps and $O(s \log s)$ other steps. The length of required molecules is $O(s)$ [11].
- Beaver (1995): Polynomial-step molecular computers compute PSPACE [2].
- Rooss and Wagner (1996): The class of problems that can be solved by Lipton's model in polynomial time is exactly P^{NP} [12].

2.2. The analysis of chemical reactions

As is discussed in the previous section, the time required to complete the chemical reactions in an experimental step is not included in the classical results. From the perspective of the computational power of molecules, however, it is important to analyze the chemical reactions, in detail. In particular, each chemical reaction should be analyzed to determine both (1) the total yield of the reaction, and the potential for generating errors. In a DNA hybridization reaction, for instance, the total amount of correctly hybridized, double stranded DNA (dsDNA) corresponds to the former, while the fraction of unexpected dsDNA corresponds to the latter.

Given that the set of chemical reactions which implement a molecular algorithm will attain equilibrium, during experimental times of interest, it is important to design the set of expected equilibrium state so that the yield is maximized. This may be achieved, in general by adjusting, through encoding strategies, the sizes of the equilibrium constants of planned (and unplanned) structures at reaction conditions of interest. In addition, it is also important to make the reaction rate large in order to shorten the time to reach the equilibrium. Even if the equilibrium constant is large, a small forward reaction rate will make the time for the overall computation long. Note that the time for a chemical reaction is never zero.

Research on the analysis of the time required for chemical reactions, in the context of molecular computing, includes the following. Kurtz analyzed the process of path formation in Adleman's experiment [1], based on chemical kinetics [10]. He showed that even for a simple graph with back edges, the time required to generate a Hamiltonian path is $\Omega(n^2)$ for n vertices [10]. For more complex graphs, the required time is much longer. In [16], Rose, et al. used a kinetic model of hairpin formation

in order to estimate the characteristic relaxation times for Whiplash PCR. Winfree has also analyzed the process of DNA tiling, using a kinetic model [23].

Errors in hybridization reactions have been well studied. In order to avoid mishybridization (i.e., errors in the hybridization reaction), it is important to adjust reaction conditions such as temperature and salt concentration, and to design “good” sequences. Rose et al. have applied an equilibrium chemistry-based analysis, in combination with a statistical thermodynamic model of equilibrium constant estimation, to assess the potential for mishybridization for both solution [13,14] and chip-based [15] molecular algorithms. One advantage of this type of analysis is the generation of a well-defined probability of error per hybridization for a given set of encodings. By interpreting the results of the error estimation process in terms of codeword set fitness, Rose et al. used this analysis to implement a genetic algorithm for producing sequence sets with very low overall error probability.

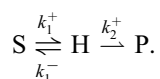
It is also possible to decrease errors simply by repeating the basic operations. For example, extraction by hybridization can be made more accurate by iterating the extraction operations (note that the yield is decreased). Karp et al. reported an exact evaluation of the number of iterations necessary to achieve an overall extraction with a per-strand error probability of less than ε [9].

2.3. Probabilistic algorithms and molecular computation

Due to the probabilistic nature of chemical reactions, it seems clear that algorithms which interpret the execution of one or more molecular processes in terms of computation will be inherently probabilistic. This implies that the results of a molecular algorithm will be meaningful only relative to some permissible error probability. Conversely, this inherent probabilistic nature also suggests that it should be natural to implement probabilistic algorithms using molecular processes. The merits of such an approach include both the massive parallelism and the randomness inherent in chemical reactions. In particular, it seems likely that the well-characterized randomness inherent in molecular reactions may be used to implement the random operations necessary for probabilistic algorithms. Thus far, the molecular computation-based probabilistic algorithms which have been reported in the literature are few. They include molecular algorithms for 3-SAT [5,7] and PAC-learning [18].

3. Hairpin formation

In this section, we estimate the total time required for (1) hairpin formation, and (2) a hairpin detection operation, which removes the detected hairpin from the reaction. We assume the following simple reaction:



In the first reaction, S denotes a DNA molecule that has not formed a hairpin structure, while H denotes the DNA hairpin. This reaction is reversible, with forward and

backward rate constants, k_1^+ and k_1^- . Note that k_1^+ is the rate constant for hairpin closure while k_1^- is that of hairpin opening.

The second reaction accomplishes the removal of molecules that have formed a hairpin structure. In the SAT Engine, two experimental techniques are employed to remove molecules which contain a hairpin. In this paper, we simply model the SAT Engine by assuming an irreversible reaction that converts the DNA hairpin, H into some product molecule, denoted by P. The forward rate constant of this reaction is assumed to be length independent, and is denoted by k_2^+ .

The equilibrium constant of K_{eq} for the first reaction has the following relationship with k_1^+ and k_1^- .

$$K_{eq} = \frac{k_1^+}{k_1^-}.$$

According to the Jacobson–Stockmayer expression [4], K_{eq} is proportional to $1/n^{1.5}$, where n is the loop length of the hairpin to be formed. In the SAT Engine, the length n of the hairpin is, in the worst case, proportional to the number of clauses. In this paper, we measure the loop length by the number of sequences representing literals. Consequently, the loop length in the worst case coincides with the total number n of clauses of the 3-SAT problem.

In general, both the forward (k_1^+) and reverse (k_1^-) rate constants for DNA hairpin formation are known to depend on the length of the hairpin loop, n [3]. However, as n becomes very large, the n -dependency of k_1^- should become negligible, since the loop size becomes irrelevant to the local process of hairpin opening at the stem n . For purposes of analyzing the asymptotic behavior obtained when the hairpin length reaches infinity, we therefore approximate k_1^- as a constant (with respect to variations in n). In this case, k_1^+ is proportional to $1/n^{1.5}$, and

$$k_1^+ = \frac{k}{n^{1.5}},$$

where k is a quantity that does not depend on n . As noted above, k_2^+ is also assumed to be independent of the hairpin loop length, n .

According to standard chemical kinetics [22] the concentrations of the single-stranded coil (C_S) and folded hairpin (C_H), and final product (C_P) at any time, t obey the following set of differential equations:

$$\dot{C}_S = -k_1^+ C_S + k_1^- C_H,$$

$$\dot{C}_H = k_1^+ C_S - (k_1^- + k_2^+) C_H.$$

Each eigenvalue, z of these differential equations should satisfy the following quadratic equation.

$$z^2 + (k_1^+ + k_1^- + k_2^+)z + k_1^+ k_2^+ = 0.$$

As solutions, we have

$$z = \frac{-(k_1^+ + k_1^- + k_2^+) \pm \sqrt{(k_1^+ + k_1^- + k_2^+)^2 - 4k_1^+ k_2^+}}{2}.$$

If $k_1^+ = 0$, we simply have

$$z = 0, -(k_1^- + k_2^+).$$

In order to examine how the first solution reaches 0 as k_1^+ reaches 0, we differentiate the expression

$$\frac{-(k_1^+ + k_1^- + k_2^+) + \sqrt{(k_1^+ + k_1^- + k_2^+)^2 - 4k_1^+ k_2^+}}{2}$$

by k_1^+ , and obtain the following:

$$\frac{1}{2} \left(-1 + (2(k_1^+ + k_1^- + k_2^+) - 4k_2^+) \frac{1}{2\sqrt{(k_1^+ + k_1^- + k_2^+)^2 - 4k_1^+ k_2^+}} \right).$$

At $k_1^+ = 0$, it is

$$-\frac{k_2^+}{k_1^- + k_2^+}.$$

Therefore, in the limit $n \rightarrow \infty$, (i.e., $k_1^+ \rightarrow 0$), the eigenvalues of the equations are $-k_1^+ k_2^+ / (k_1^- + k_2^+)$ and $-(k_1^- + k_2^+)$. The first eigenvalue may be written as

$$-\frac{k_2^+ k}{(k_1^- + k_2^+) n^{1.5}}$$

since $k_1^+ = k/n^{1.5}$. For large n , the concentration, C_S of single-stranded coils therefore decreases according to the following expression:

$$A \exp \left(-\frac{k_2^+ k t}{(k_1^- + k_2^+) n^{1.5}} \right) + B \exp(-(k_1^- + k_2^+) t).$$

Note that A and B are appropriate constants. Since the second term quickly reaches 0, the first term becomes dominant for large n . In fact, in the limit $k_1^+ \rightarrow 0$, A reaches the initial concentration of single-stranded coils, and B reaches 0, if we assume that the initial concentration of folded hairpins is 0. Therefore,

$$\exp \left(-\frac{k_2^+ k t}{(k_1^- + k_2^+) n^{1.5}} \right)$$

represents the ratio of C_S at time t to its initial value.

In the SAT Engine, the concentration of unfolded ssDNAs (single-stranded coils), C_S is proportional to the probability that an inconsistent assignment cannot be detected. It is therefore the probability of false positives, i.e., the probability that an unsatisfiable Boolean expression is judged to be satisfiable.

If we are given ε_0 as the permissible error probability per strand for false positives, we have the following requirement:

$$\exp\left(-\frac{k_2^+ kt}{(k_1^- + k_2^+)n^{1.5}}\right) < \varepsilon_0.$$

This is equivalent to

$$t > \frac{(k_1^- + k_2^+)n^{1.5}}{k_2^+ k} \ln\left(\frac{1}{\varepsilon_0}\right).$$

We can therefore conclude that the time required for hairpin formation is proportional to $n^{1.5}$ and $\ln(1/\varepsilon_0)$.

The above arguments apply to each molecule that should form a hairpin. In order to estimate the overall time of the algorithm, we must take the number of molecules into account, because all the molecules with possible hairpins should be eventually removed. We estimate the number of necessary molecules in the next section.

It is important to note that a remaining problem which influences the probability of false negatives, in practice, is the potential for hairpin formation due to unplanned mishybridizations. Thanks to the recent development of both combinatorial [6] and chemistry-based [13–15] techniques for producing high-fidelity DNA encodings, however this effect can be neglected, and the encoded hairpins are assumed to be of sufficiently high fidelity to avoid substantial mismatch hybridization.

4. Random generation of assignments

In this section, we analyze the process of randomly generating the library of ssDNA molecules which encodes the set of all literal assignments from the SAT instance of interest. We first discuss a stepwise method for the generation of the random library, and then consider a simpler implementation, where the library is generated in a single step. Note that Sakamoto et al. use double stranded DNA molecules in this process. Following the completion of random generation, double stranded DNAs are denatured, and the resulting ssDNAs are assayed for the potential to undergo hairpin formation.

4.1. Stepwise generation

For each clause we synthesize M molecules, where $M/3$ copies are encoded to represent each literal contained in the clause. The M molecules from the first clause are then mixed with those from the second clause. We assume that in the encoding process, literals from the second clause have been encoded to contain a sticky end that is Watson–Crick complementary to the sticky end of the literals from the first clause.

Following hybridization, the concatenated molecules are ligated. The ligated mixture is then mixed with the M molecules from the third clause, which are again assumed to have been encoded with a sticky end that is complementary to a sticky end of the literals from the second clause. Following hybridization, the concatenated molecules are again ligated. This process is continued for all remaining clauses. The time for the entire process is clearly proportional to the number of clauses.

Here, we assume that there is one satisfying assignment, and then compute the probability that this assignment is generated by the above process. It is also assumed that for each clause, there is only one correct choice per literal for generating the assignment. In each step, a partial assignment is hybridized to one of the three literals of the next clause with some probability p that is nearly one-third. At the end of this section, we argue that p can be made as close to one-third as is desired.

Recall that the hairpin loop size is, in the worst case, proportional to the number of clauses, and n denotes both the number of clauses and the hairpin loop size in the worst case.

We regard the process of generating a satisfying assignment is that of making n choices with probability p . Since we begin with M molecules, we make M trials. The probability that a satisfying assignment is not generated is therefore the probability that all M trials fail, i.e.,

$$(1 - p^n)^M.$$

Strictly speaking, this is an upper bound of the probability, because the M trials are not independent. For this probability to be smaller than ε_2 , we should have

$$(1 - p^n)^M < \varepsilon_2.$$

We therefore have $M \ln(1/(1 - p^n)) > \ln(1/\varepsilon_2)$. Application of the approximation, $\ln(1/(1 - p^n)) \approx p^n$, yields

$$M > \frac{\ln(1/\varepsilon_2)}{p^n}.$$

Let $\alpha > 0$ be defined such that

$$p = \frac{1}{3 + \alpha}.$$

In this case, we finally have

$$M > (3 + \alpha)^n \ln \left(\frac{1}{\varepsilon_2} \right).$$

In summary, in order to make the probability that a satisfying assignment is not generated, even though such an assignment exists smaller than ε_2 , we require a total number of molecules that is proportional to $(3 + \alpha)^n$ and $\ln(1/\varepsilon_2)$, where n is the number of clauses. The required time is proportional to n . Since ε_2 is the probability that a Boolean expression will be judged to be unsatisfiable, even though a satisfying set of assignments exists, it corresponds to the overall probability of a false negative.

If the hybridization reactions which implement each step of the assembly process were capable of reaching completion, α could be made to approach 0 (so that $p = 1/3$) prior to each round of ligation. Due to the reversibility of DNA hybridization, however, some fraction of DNA molecules is always expected to assume a random coil configuration. α may be made small by making hybridization irreversible. This may be accomplished by performing the hybridization and ligation reactions in parallel in which case α should decrease exponentially with time.

4.2. One-step generation

In order to generate the random library in a single step, we again prepare $M/3$ copies of each literal of each clause, and place a total of nM molecules into the reaction tube. Molecules representing assignments are then generated spontaneously. Note that sticky ends that connect literals from the i th clause and literals from the $(i + 1)$ th clause should be distinct for each i .

Consider the probability that a molecule representing a literal in the i th clause comes into contact with a molecule in the $(i + 1)$ th clause. Since there are nM molecules, where n is the number of clauses, the probability of contact will increase as $1/n$. In order to keep the probability constant, we must therefore increase the applied reaction time in a manner proportional to n . The required time for the SAT Engine, including assembly, is therefore also proportional to the number of clauses, n .

5. Overall estimation

The time required for completion of the overall process of hairpin formation, with combined error probabilities less than ε_1 and ε_2 , is estimated by explicitly taking the number of molecules into account. If almost all assignments are unsatisfiable, then nearly all of all the M molecules should form a hairpin. From Section 3, the average number of molecules that fail to form a hairpin (and be removed from the reaction) after time t is given by

$$M \exp \left(-\frac{k_2^+ kt}{(k_1^- + k_2^+) n^{1.5}} \right).$$

By substituting $M = (3 + \alpha)^n \ln(1/\varepsilon_2)$ into this expression, and setting the average number to be $< \varepsilon_1$, we have

$$(3 + \alpha)^n \ln \left(\frac{1}{\varepsilon_2} \right) \exp \left(\frac{-k_2^+ kt}{(k_1^- + k_2^+) n^{1.5}} \right) < \varepsilon_1.$$

By taking logarithm of both sides, we finally have

$$\frac{k_2^+ kt}{k_1^- + k_2^+} > n^{1.5} \left(\ln \left(\frac{1}{\varepsilon_1} \right) + \ln \left(\ln \left(\frac{1}{\varepsilon_2} \right) \right) \right) + n^{2.5} \ln(3 + \alpha).$$

In conclusion, if ε_1 and ε_2 denote the probability of false positives and false negatives, respectively, the required time is proportional to

$$n^{1.5} \left(\ln \left(\frac{1}{\varepsilon_1} \right) + \ln \left(\ln \left(\frac{1}{\varepsilon_2} \right) \right) \right) + n^{2.5} \ln(3 + \alpha),$$

and the number of necessary molecules is proportional to $(3 + \alpha)^n \ln(1/\varepsilon_2)$.

6. Concluding remark

In this paper, the time complexity of the SAT Engine was estimated for a given pair of error probabilities, ε_1 (the probability of a false positive) and ε_2 (the probability of a false negative). Although the result is not impressive, we consider it to be first step towards analyzing the complexity of molecular computation from the following two perspectives. The first is the analysis of molecular algorithms as probabilistic algorithms. The second is the explicit inclusion of the efficiency of molecular reactions, in a way which should be independent of the current implementation of the experimental techniques.

Acknowledgements

This work was supported by the Japan Society for the Promotion of Science under grant JSPS-RFTF 96I00101 and a JSPS Grant-In-Aid, and also by CREST of JST (Japan Science and Technology Corporation).

References

- [1] L.M. Adleman, Molecular computation of solutions to combinatorial problems, *Science* 266 (1994) 1021–1024.
- [2] D. Beaver, A universal molecular computer, *DNA Based Computers*, DIMACS Series in Discrete Mathematics Theoret. Comput. Sci. 27 (1996) 29–36.
- [3] G. Bonnet, O. Krichevsky, A. Libshaber, Kinetics of conformational fluctuations in DNA hairpin-loops, *Proc. Nat. Acad. Sci.*, Vol. 95, pp. 8602–8606.
- [4] C.R. Cantor, P.R. Schimmel, *Biophysical Chemistry, Part III: The Behavior of Biological Macromolecules*, W. H. Freeman and Company, New York, 1980, p. 1207.
- [5] K. Chen, V. Ramachandran, A space-efficient randomized DNA algorithm for k -SAT, *DNA6*, Sixth Internat. Meeting on DNA Based Computers, Leiden Center for Natural Computing, June 13–17, 2000, pp. 171–181.
- [6] R. Deaton, M. Garzon, R. Murphy, J. Rose, D. Franceschetti, S. Edward Stevens Jr., Reliability and efficiency of a DNA-based computation, *Phys. Rev. Lett.* Vol. 80, pp. 417–420.
- [7] S. Díaz, J.L. Esteban, M. Ogihara, A DNA-based random walk method for solving k -SAT, *DNA6*, Sixth Internat. Meeting on DNA Based Computers, Leiden Center for Natural Computing, June 13–17, 2000, pp. 181–191.
- [8] M. Hagiya, M. Arita, D. Kiga, K. Sakamoto, S. Yokoyama, Towards parallel evaluation and learning of Boolean μ -formulas with molecules, *DNA Based Computers III*, DIMACS Series in Discrete Mathematics; Theoret. Comput. Sci. 48 (1999) 57–72.

- [9] R. Karp, C. Kenyon, O. Waarts, Error-resilient DNA computations, *Seventh ACM-SIAM Symp. on Discrete Algorithms*, 1996, pp. 458–467.
- [10] S.A. Kurtz, S.R. Mahaney, J.S. Royer, J. Simon, Active transport in biological computing, *DNA Based Computers II*, DIMACS Series in Discrete Mathematics; Theoret. Comput. Sci. 44 (1999) 171–179.
- [11] J.H. Reif, Parallel molecular computation, *Seventh Annual ACM Symp. on Parallel Algorithms and Architectures (SPAA'95)*, 1995, pp. 213–223.
- [12] D. Roß, K.W. Wagner, On the power of DNA-computing, *Inform. and Comput.* 131 (1996) 95–109.
- [13] J.A. Rose, R.J. Deaton, The fidelity of annealing-ligation: a theoretical analysis, *DNA Computing, Lecture Notes in Computer Science*, Vol. 2054, Springer, Berlin, 2000, pp. 231–241.
- [14] J.A. Rose, R.J. Deaton, D.R. Franceschetti, M. Garzon, S.E. Stevens Jr., A statistical mechanical treatment of error in the annealing biostep of DNA computation, *Proc. Genetic and Evolutionary Computation Conf. (GECCO'99)*, pp. 1829–1834.
- [15] J.A. Rose, R.J. Deaton, M. Hagiya, A. Suyama, The fidelity of the tag-antitag system, *Preliminary Proc. Seventh Internat. Meeting on DNA Based Computers, DNA7*, pp. 302–310.
- [16] J.A. Rose, R.J. Deaton, M. Hagiya, A. Suyama, An equilibrium analysis of the efficiency of an autonomous molecular computer, *Phys. Rev. E*, in press.
- [17] J.A. Rose, M. Hagiya, R.J. Deaton, A. Suyama, A DNA-based in vitro genetic program, *J. Biol. Phys.*, in press.
- [18] Y. Sakakibara, Solving Computational learning problems of Boolean formulae on DNA computers, *DNA6, Sixth Internat. Meeting on DNA Based Computers, Leiden Center for Natural Computing*, June 13–17, 2000, pp. 193–204.
- [19] K. Sakamoto, H. Gouzu, K. Komiya, D. Kiga, S. Yokoyama, T. Yokomori, M. Hagiya, Molecular computation by DNA hairpin formation, *Science* 288 (2000) 1223–1226.
- [20] K. Sakamoto, D. Kiga, K. Komiya, H. Gouzu, S. Yokoyama, S. Ikeda, Hiroshi, Sugiyama, M. Hagiya, State transitions by molecules, *Biosystems* 52 (1999) 81–91.
- [21] U. Schöning, A probabilistic algorithm for k -SAT and constraint satisfaction problems, *Proc. 40th Symp. on Foundations of Comput. Sci.*, IEEE Computer Society Press, Silver Spring, MD, 1999, pp. 410–414.
- [22] J.I. Steinfeld, J. Francisco, W.L. Hase, *Chemical Kinetics and Dynamics*, Prentice-Hall, Englewood Cliffs, NJ, 1989.
- [23] E. Winfree, Simulations of computing by self-assembly, *Preliminary Proc. Fourth Internat. Meeting on DNA Based Computers*, June 15–19, 1998, University of Pennsylvania, pp. 213–239, 1998. Also in Erik Winfree's Ph.D. Thesis, *Algorithmic Self-Assembly of DNA*, California Institute of Technology, May 19, 1998.